Precision UI Hybrid Automation Framework

Project Overview

Project Name: Precision UI Hybrid Automation Framework

Assignment: Veeva Fresh Graduate Assignment – Web Test Automation

Team size: 2

Team Members:

- Venkateswara Rao Bhimasingi
- Sai Ganesh Immidisetti

GitHub Link: https://github.com/Venkateswararaobhimasingi/precision-ui-hybrid-automation-framework.git

Duration: March 2026 – April 2026

Website Under Test: https://www.iplt20.com/

Abstract

The Precision UI Hybrid Automation Framework is a scalable and efficient test automation solution designed to validate the UI functionalities of the IPL website. It is built using Java, Selenium WebDriver, TestNG, and Maven, following a hybrid approach that combines the Page Object Model (POM) and data-driven testing. The framework automates key functionalities such as footer link validation, team details verification, points table accuracy, and search operations. It supports cross-browser execution and structured test grouping like smoke and regression for better coverage. The architecture is layered into Base, Page Objects, Test Cases, and Utilities to ensure modularity and maintainability. Test execution is managed through TestNG, flowing from initialization and setup to execution, validation, and teardown. Utilities like wait handling, Excel integration, and data providers enhance flexibility and reliability. Detailed reporting with Extent Reports, logs, and screenshots ensures clear visibility and easier debugging, making the framework industry-ready and extensible.

Table of Contents

Section	Page No.
Introduction	1
Objectives	2
Technical Stack	3
Roles & Responsibilities	4
Framework Architecture	6
Workflow	10
Detailed Test Cases	12
Reporting & Logging	14
Report Screenshots	15
Conclusion	16

Introduction

Purpose of the Framework

The purpose of the Precision UI Hybrid Automation Framework is to automate the testing of web-based user interface functionalities of the IPL website in a reliable and efficient manner. The framework is designed to reduce manual testing effort, improve test coverage, and ensure consistent validation of application behavior across different scenarios.

It provides a structured approach to automation by using Selenium WebDriver, TestNG, and the Page Object Model (POM), enabling testers to write clean, reusable, and maintainable test scripts. Additionally, it supports cross-browser execution and data-driven testing, making it suitable for real-world industry applications.

Scope of Testing

The scope of this framework includes automation of key UI functionalities of the IPL website. The testing focuses on validating core user interactions and ensuring that important features work as expected.

The following areas are covered under automation:

- Validation of footer links and navigation
- Verification of team details and related information
- Validation of points table data such as rankings and scores
- Testing of search functionality for teams and players

The framework primarily focuses on functional UI testing and does not include backend/API testing or performance testing.

Key Features

The framework is designed with several important features that make it robust and industry-ready:

- Hybrid Framework Design combining POM and Data-Driven Testing
- Page Object Model (POM) for separation of test logic and UI elements
- Cross-Browser Testing support (Chrome, Edge, Firefox)
- Data-Driven Testing using Excel and TestNG DataProviders
- Reusable Components through base classes and utilities
- Explicit Wait Handling to improve test stability
- Detailed Reporting using Extent Reports with logs and screenshots
- Modular Architecture for easy maintenance and scalability
- TestNG Integration for structured test execution and grouping

Objectives

Automation Goals

The primary automation goal of the Precision UI Hybrid Automation Framework is to minimize manual testing efforts by automating repetitive and critical UI test scenarios of the IPL website. The framework aims to ensure faster test execution, improved accuracy, and consistent results across multiple test runs.

It is designed to:

- Automate end-to-end user workflows
- Enable cross-browser testing for better coverage
- Support data-driven testing for multiple input scenarios
- Provide a scalable and reusable automation solution

Functional Validations

The framework focuses on validating key functionalities of the IPL website to ensure correct behavior from a user perspective. The main functional validations include:

- Verifying that all footer links are present and navigate correctly
- Ensuring accurate display of team details such as players and statistics
- Validating points table data including rankings, matches, and scores
- Checking the correctness of search functionality for teams and players

These validations ensure that the application meets functional requirements and provides a seamless user experience.

Reporting & Logging

The framework includes a robust reporting and logging mechanism to track test execution and results effectively.

- Extent Reports are used to generate detailed HTML reports with step-by-step execution logs
- Each test step is logged using:
    - info() → for execution steps
    - pass() → for successful validations
    - fail() → for failed steps
- Screenshots are captured on failure to assist in debugging
- Logs provide complete traceability of test execution

This reporting and logging system helps testers and developers quickly identify issues, analyze failures, and improve overall test reliability.

Technical Stack

- **Programming Language:** Java
- **Automation Tool:** Selenium WebDriver
- **Test Engine:** TestNG (Annotations, Assertions, Parallel Execution)
- **Build Tool:** Maven (Dependency Management)
- **Reporting:** Extent Reports (HTML Dashboard with Screenshots)
- **Logging:** Log4j2 (Structured Logging and Debugging)
- **Data Handling:** Apache POI (Excel Integration)
- **Design Pattern:** Page Object Model (POM)

Roles & Responsibilities

The project was developed collaboratively with clear ownership of test modules, while maintaining shared responsibility for framework design and core components. Each team member contributed to specific functional areas as well as the overall architecture.

Venkateswara Rao Bhimasingi

- Designed and implemented Footer Links Validation test scenario
    - Implemented dynamic footer section validation using configuration-based inputs.
    - Developed logic to iterate through multiple sections and verify link presence.
- Developed Team Details Validation module
    - Implemented validation for team logo visibility using dynamic locators
    - Automated hover interactions using Actions class
    - Built logic to extract and validate winning years data
    - Handled edge cases such as teams with no titles (NA scenario)
- Implemented data-driven testing
    - Integrated Excel-based test data using DataProvider
    - Designed validation logic for comparing expected vs actual team data
- Contributed to Page Object design for Home and Team modules
- Added detailed logging (Log4j2) for step-by-step execution tracking
- Ensured proper assertion handling and validation flow

Sai Ganesh Immidisetti

- Designed and implemented Points Table Validation module
    - Built dynamic XPath-based logic to fetch team-specific data
    - Extracted match statistics (played, won, lost, no result, points)
    - Implemented validation logic to verify points calculation consistency
- Developed Search Functionality Validation
    - Automated search interaction (input, filter selection, execution)
    - Implemented validation of search results against expected output
    - Handled UI interactions such as search dropdown and result extraction
- Created Page Object methods for points table and search features
- Focused on data extraction, validation logic, and assertions
- Integrated logging for debugging and execution traceability

Shared Contributions

Both team members collaboratively worked on core framework components and design:

- Developed the Base Layer
    - Browser initialization using TestNG parameters (cross-browser support)
    - Configuration handling via properties file
    - Screenshot capture for failure scenarios
- Designed the framework using Page Object Model (POM)
    - Ensured separation of test logic and UI interactions
    - Promoted reusable and maintainable code structure
- Built Utility Layer:
    - Wait handling for dynamic elements
    - Excel utility for data-driven testing
    - DataProvider integration
    - Extent Report integration for reporting
- Implemented Extent Reports Listener
    - Captured test execution results (pass/fail/skip)
    - Attached screenshots on failures
    - Generated consolidated HTML reports
- Integrated Log4j2 logging framework
    - Enabled structured logging (info, debug, error)
    - Supported debugging and execution traceability
- Configured TestNG Execution Flow
    - Multi-browser execution (Chrome, Edge, Firefox)
    - Group-based execution (smoke, regression)
    - Suite-level reporting integration
- Performed testing, debugging, and optimization
    - Handled synchronization issues using explicit waits
    - Ensured framework stability and reliability

Framework Architecture

The Precision UI Hybrid Automation Framework is designed using a layered architecture that ensures modularity, reusability, and maintainability. The framework follows the Page Object Model (POM) design pattern, which separates test logic from UI interactions, making the code clean, scalable, and easy to manage.

The architecture is divided into four major components: Base Package, Page Objects, Test Cases, and Utilities.

Base Package

The Base Package acts as the foundation layer of the framework and is responsible for initializing and managing core functionalities required across the project.

- BaseTest
    - Handles browser setup and teardown using TestNG annotations
    - Supports cross-browser execution (Chrome, Edge, Firefox)
    - Loads configuration properties such as application URL
    - Integrates logging using Log4j for execution tracking
    - Provides screenshot capture functionality for debugging failures
- BasePageObject
    - Acts as a parent class for all page objects
    - Initializes WebDriver, WebDriverWait, JavaScriptExecutor, and Actions
    - Uses PageFactory for element initialization
    - Provides reusable setup and common functionality for UI interactions

Page Objects

This layer implements the Page Object Model (POM), where each class represents a specific page of the application. It contains web element locators and methods to interact with them, ensuring separation of concerns.

- HomePageObject
    - Handles homepage functionalities such as navigation, footer links, search operations, and cookie handling
    - Provides reusable methods like scrolling, searching, and navigating between sections
- PointsTablePageObject
    - Handles extraction and validation of points table data
    - Provides methods to retrieve match statistics such as matches played, wins, losses, and total points
    - Uses dynamic locators for flexible data handling

- TeamDetailsPageObject
    - Handles team-related functionalities such as logo validation and retrieving winning years
    - Uses hover actions for dynamic UI interactions
    - Ensures stable execution using wait mechanisms

Test Cases

The Test Cases layer contains actual test scenarios that validate application functionality. Each test case follows a structured approach: initialization → action → validation.

- Footer Links Validation
    - Validates the presence and correctness of footer links across different sections
- Team Details Validation
    - Verifies team logo visibility and winning years
    - Uses data-driven testing for multiple teams
- Points Table Validation
    - Validates points table data and ensures calculation correctness
- Search Functionality Validation
    - Tests search feature behavior and validates search results

All test cases:

- Use TestNG annotations for execution
- Perform assertions for validation
- Log execution steps for traceability

Utilities

The Utilities package provides reusable support functionalities that enhance the framework's efficiency, stability, and scalability.

- Wait Utilities
    - Handles explicit waits for element presence, visibility, and interaction readiness
    - Prevents flaky test failures caused by timing issues
    - Ensures elements are properly loaded before performing actions
- Excel Utility
    - Reads test data from Excel files using Apache POI
    - Supports external data management, avoiding hardcoded values
    - Enables flexible and scalable test data handling

- DataProviderUtilities
    - Supplies test data to test cases using TestNG DataProvider
    - Integrates with ExcelUtility to enable data-driven testing
    - Allows execution of the same test with multiple datasets
- ExtentReportManager
    - Generates detailed HTML reports for test execution
    - Logs step-by-step execution status (pass, fail, skip)
    - Captures and attaches screenshots on failure
    - Provides a clear and shareable execution summary

Architecture Diagram

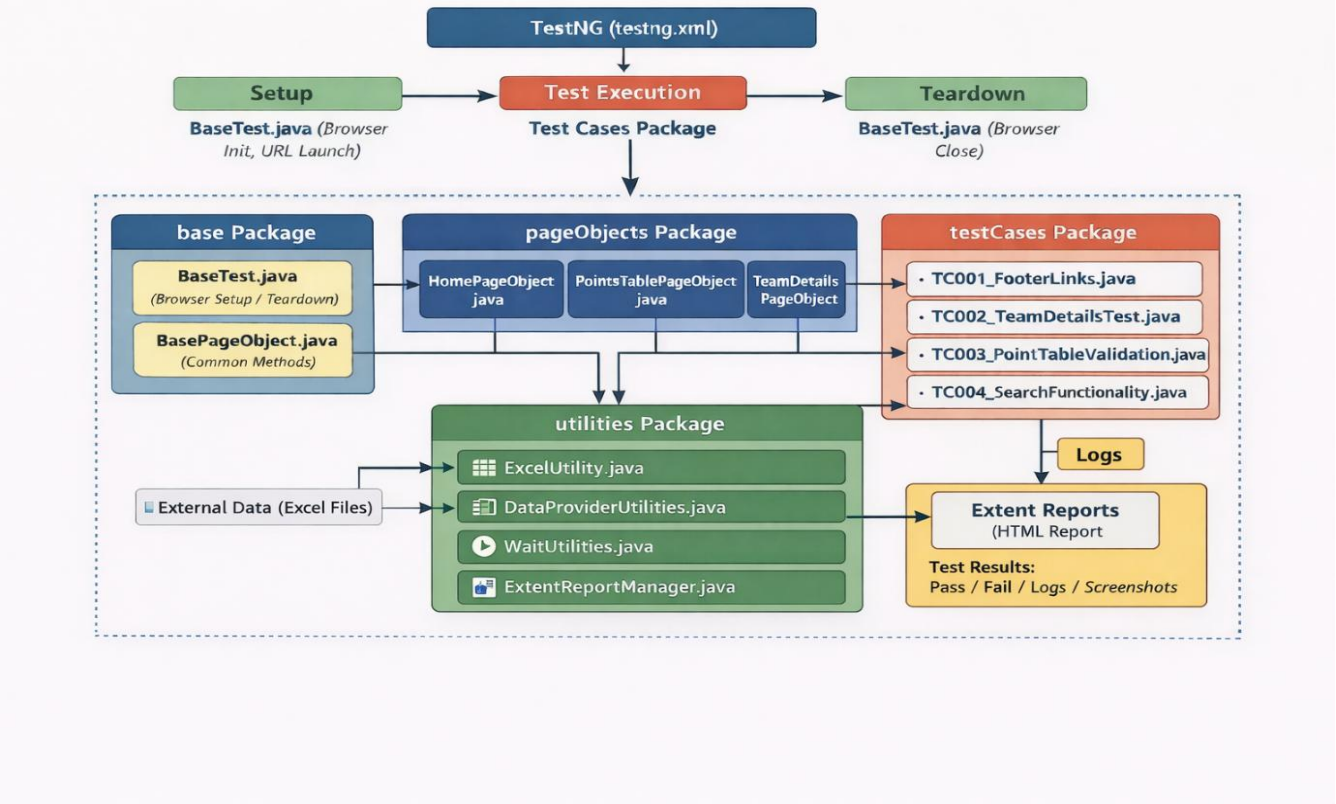

Fig: Architecture diagram- Precision UI Hybrid Automation Framework

Execution

The test execution in the framework is controlled using the TestNG configuration file (testng.xml), which acts as the central entry point for running all automated test cases.

The framework supports both single-browser execution and multi-browser parallel execution using separate suite configurations.

Execution Configuration

- The Master Suite is configured to run tests across multiple browsers:
    - Chrome
    - Edge
    - Firefox
- Each browser is defined as a separate <test> block with:
    - Browser parameter (browser)
    - Included test groups (smoke, regression)
    - List of test classes to execute

Key Features of Execution

- Cross-Browser Execution
  Tests are executed on multiple browsers using parameterization.
- Parallel Execution
  Thread count is defined to enable faster execution.
- Group-Based Execution
  Test cases are filtered using groups:
    - Smoke → Critical validations
    - Regression → Full test coverage
- Listener Integration
    - Extent Report Manager is attached at suite level
    - Captures execution results and generates reports

Execution Flow

1. TestNG reads the suite configuration
2. Browser parameter is passed to the Base setup
3. WebDriver is initialized based on browser type
4. Selected test groups are executed
5. Test classes run sequentially/parallel
6. Results are captured and sent to reporting
7. Browser is closed after execution

Workflow

The framework follows a structured workflow that ensures smooth interaction between different layers such as Base, Test Cases, Page Objects, and Utilities.

Step-by-Step Workflow

1. TestNG Initialization
    - Reads testng.xml
    - Identifies test suite, browser, and groups
2. Setup Phase (Base Layer)
    - Browser is launched using Base setup
    - URL is loaded from configuration file
    - Logger is initialized
    - Page Object instances are created
3. Test Execution Phase
    - Test cases are triggered based on groups
    - Each test interacts with the application via Page Objects
    - Actions such as click, search, navigation, and validation are performed
4. Page Object Interaction
    - Page Objects handle all UI operations
    - Web elements are located using locators
    - Reusable methods perform actions on UI
5. Utility Support
    - Wait utilities ensure synchronization
    - Excel utilities provide test data
    - Data providers feed multiple datasets
    - Reporting utilities capture execution details
6. Validation Phase
    - Assertions validate expected vs actual results
    - Failures are logged with detailed information
7. Reporting & Logging
    - Logs are generated for each step
    - Screenshots are captured on failure
    - Extent Reports generate final execution report

8. Teardown Phase
    - Browser is closed
    - Execution is finalized
    - Report is generated and stored

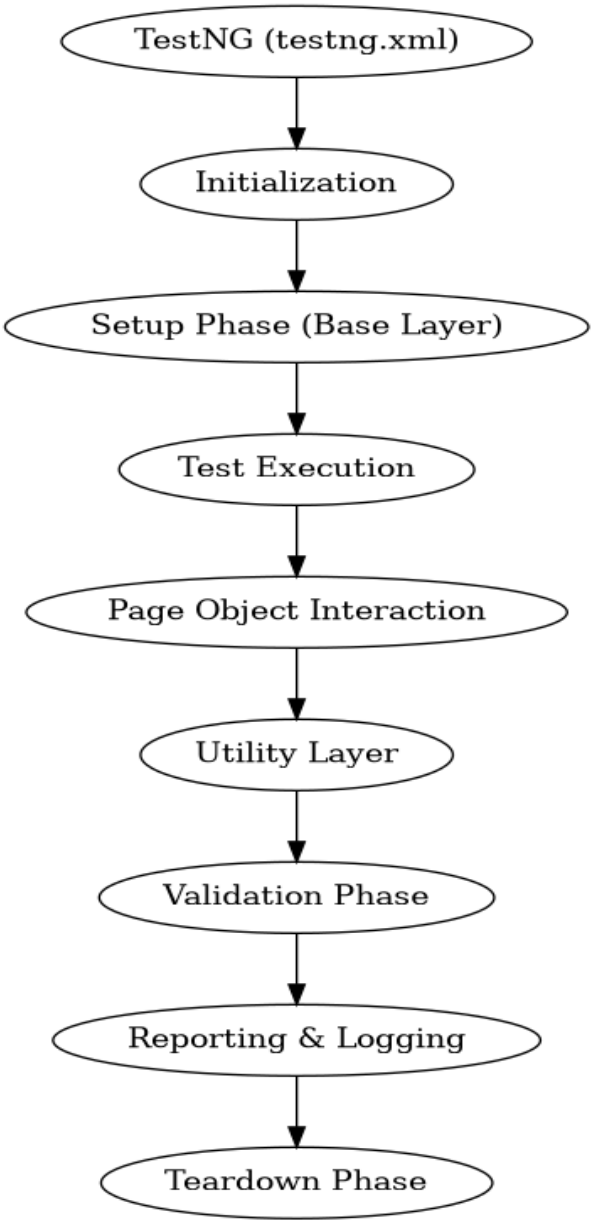

Fig: Workflow diagram- Precision UI Hybrid Automation Framework

Detailed Test Cases

This section describes the key automated test scenarios implemented in the framework. Each test case is designed to validate critical functionalities of the application using structured steps and assertions.

1. Footer Links Validation

Objective: To verify that all footer sections contain valid and visible links.

Test Steps:

1. Launch the application in the browser
2. Handle cookie popup if displayed
3. Scroll to the bottom of the page
4. Retrieve all configured footer section names from properties file
5. For each section:
    - Extract all links under that section
    - Validate that links are present

Validation:

- Ensure each footer section contains at least one link
- Fail the test if any section has missing links

Expected Result: All footer sections should contain valid links.

2. Team Details Validation

Objective: To validate team-related information such as logo visibility and winning years.

Test Steps:

1. Navigate to the Teams page
2. Handle cookie popup
3. For each team (using data-driven approach):
    - Verify team logo is displayed
    - Hover over the team logo
    - Retrieve winning years information

Validation:

- Compare actual logo visibility with expected value
- Validate winning years with expected data from Excel
- Handle cases where no titles are present (NA scenario)

Expected Result: Team logo should be displayed correctly, and winning years should match expected data.

3. Points Table Validation

Objective: To validate the correctness of points table data.

Test Steps:

1. Navigate to Points Table page
2. Handle cookie popup
3. Retrieve team at Rank 1
4. Extract match statistics:
    - Matches Played
    - Matches Won
    - Matches Lost
    - No Result
    - Points

Validation:

- Verify:
    - Total matches = Wins + Losses + No Result
    - Points calculation = (2 × Wins) + No Result

Expected Result: Points table data should be consistent and correctly calculated.

4. Search Functionality

Objective: To validate search functionality of the application.

Test Steps:

1. Click on search icon
2. Handle cookie popup
3. Enter search keyword (e.g., "Auction 2026")
4. Select search type as "News"
5. Click search button
6. Retrieve search result

Validation:

- Compare the displayed result with expected output

Expected Result: Search results should match the expected content based on input keyword.

Reporting & Logging

The framework implements a robust reporting and logging mechanism to provide clear visibility into test execution, facilitate debugging, and ensure traceability of test results.

Extent Reports

The framework uses Extent Reports to generate detailed and interactive HTML reports for test execution.

Key Features:

- Generates a visually rich HTML dashboard after execution
- Displays test status (Pass, Fail, Skip) for each test case
- Captures execution time and test steps
- Includes system information such as browser, OS, and execution groups
- Provides a consolidated report for all executed test cases

Screenshots on Failures

To enhance debugging, the framework captures screenshots automatically whenever a test case fails.

Implementation Details:

- Screenshots are captured using WebDriver's screenshot capability
- Each screenshot is saved with a timestamp for uniqueness
- The file path is attached directly to the Extent Report

Structured Logs

The framework uses a logging mechanism to track execution flow and capture detailed runtime information.

Key Features:

- Logs are generated using a logging framework
- Maintains different log levels:
    - INFO – General execution steps
    - DEBUG – Detailed internal information
    - ERROR – Failures and exceptions
- Logs are stored in external files for future reference
- Supports rolling log files for better management

Report Screenshots:

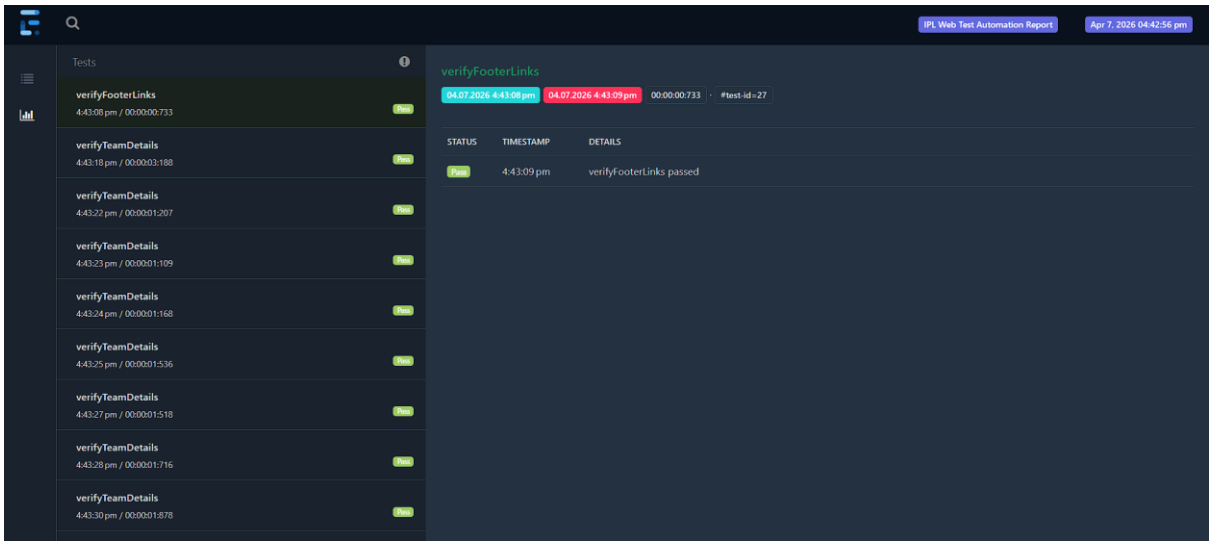

Fig: Extent Report Showing Test Case Execution Result

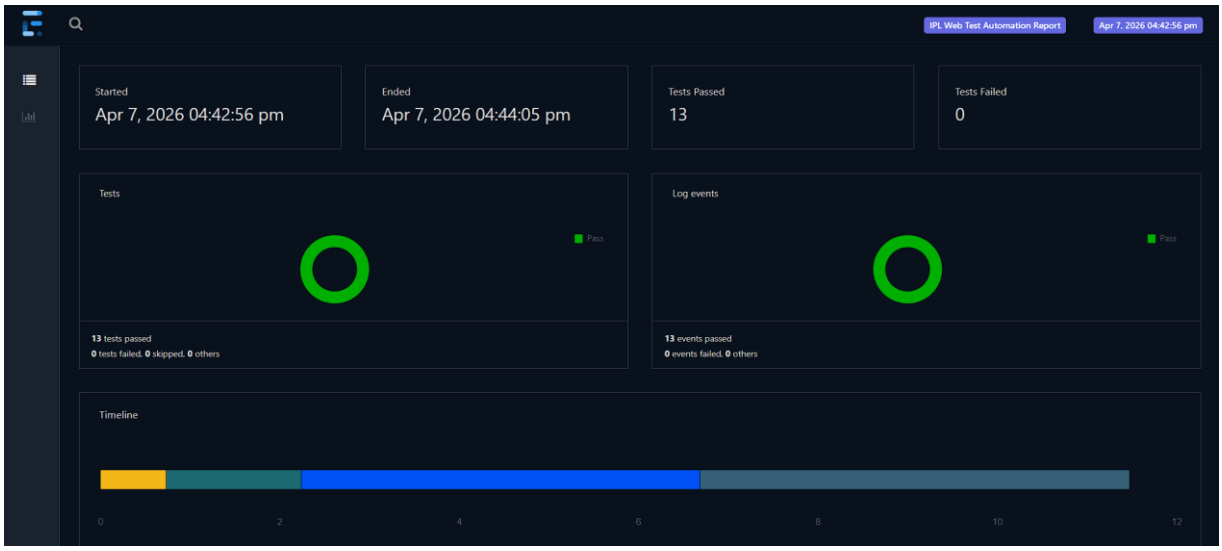

Fig: Extent Report Analysis Dashboard

Conclusion

The Precision UI Hybrid Automation Framework is a well-structured and scalable solution designed to automate web application testing efficiently. By leveraging modern tools and best practices, the framework ensures high reliability, maintainability, and reusability. The implementation of the Page Object Model (POM) enables a clear separation between test logic and UI interactions, making the codebase easy to manage and extend. Integration with TestNG allows flexible test execution through grouping, parameterization, and cross-browser support.

Key features such as data-driven testing, structured logging, and detailed reporting using Extent Reports enhance the overall quality of test execution. The inclusion of screenshots on failures and comprehensive logs further simplifies debugging and analysis. The framework successfully validates critical functionalities such as footer links, team details, points table accuracy, and search operations, ensuring the stability and correctness of the application under test.

Overall, this automation framework provides a strong foundation for real-world testing scenarios and can be easily extended to support additional test cases, parallel execution, and CI/CD integration in the future.